Question 14: Develop a SystemC model simulating a simple embedded controller and analyze timing behavior.

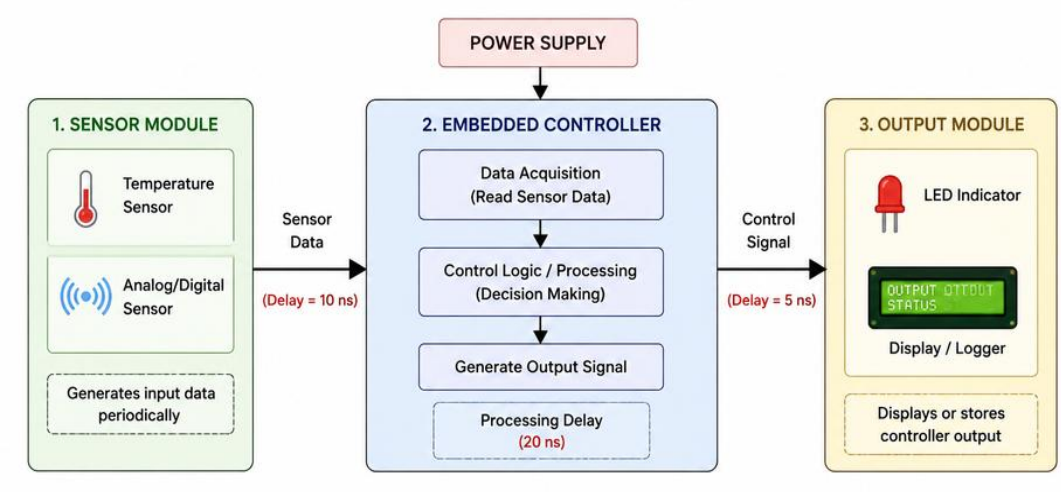
Problem Understanding

- Understand the requirement of creating a **SystemC-based embedded controller model**.
- The controller should:
    - Receive input data from sensors.
    - Process data using control logic.
    - Generate output signals.
    - Analyze execution timing and delays.

SystemC Modules

1. **Sensor Module**
    - Generates input signals periodically.
2. **Controller Module**
    - Reads sensor data.
    - Executes decision-making logic.
    - Produces control output.
3. **Output Module**
    - Displays/stores controller output.

System Architecture:



Timing Analysis

Measure:

- Sensor sampling time
- Processing delay
- Output response time
- Total execution latency

Example:

Sensor Delay = 10 ns

Processing Time = 20 ns

Output Delay = 5 ns

Total Response Time = Sensor Delay + Processing Delay + Output Delay

= 10 + 20 + 5

= 35 ns

Time (ns)	Sensor	LED
0	20	OFF
10	70	ON
20	45	OFF

SystemC Implementation:

```
#include <systemc.h>
SC_MODULE(Sensor)
{
    sc_out<int> sensor_out;

    void generate_data()
    {
        int value = 20;

        while (true)
        {
            sensor_out.write(value);

            cout << sc_time_stamp()
                 << " -> Sensor Value = "
                 << value << endl;

            value += 20;

            if (value > 100)
                value = 20;

            wait(10, SC_NS);
        }
    }

    SC_CTOR(Sensor)
    {
        SC_THREAD(generate_data);
    }
};
SC_MODULE(Controller)
{
    sc_in<int> sensor_in;
    sc_out<int> control_out;

    void process_data()
    {
        while (true)
        {
            int data = sensor_in.read();

            if (data >= 60)
                control_out.write(1);
            else
                control_out.write(0);

            cout << sc_time_stamp()
                 << " -> Controller Processed = "
                 << data
                 << "  Output = "
                 << control_out.read()
                 << endl;

            wait(20, SC_NS);
        }
    }

    SC_CTOR(Controller)
    {
        SC_THREAD(process_data);
    }
};
SC_MODULE(Output)
{
    sc_in<int> control_in;

    void display()
    {
        while (true)
        {
            cout << sc_time_stamp()
                 << " -> Output Signal = "
                 << control_in.read()
                 << endl;

            wait(5, SC_NS);
        }
    }
```

```
    SC_CTOR(Output)
    {
        SC_THREAD(display);
    }
};
int sc_main(int argc, char* argv[])
{
    sc_signal<int> sensor_signal;
    sc_signal<int> output_signal;

    Sensor sensor("Sensor");
    Controller controller("Controller");
    Output output("Output");

    sensor.sensor_out(sensor_signal);

    controller.sensor_in(sensor_signal);
    controller.control_out(output_signal);

    output.control_in(output_signal);

    cout << "===== SystemC Embedded Controller Simulation ====="
         << endl;

    sc_start(100, SC_NS);

    cout << "===== Simulation Finished =====" << endl;

    return 0;
}
```

Output:

```
===== SystemC Embedded Controller Simulation =====
0 s -> Sensor Value = 20
0 s -> Controller Processed = 0  Output = 0
0 s -> Output Signal = 0
5 ns -> Output Signal = 0
10 ns -> Sensor Value = 40
10 ns -> Output Signal = 0
15 ns -> Output Signal = 0
20 ns -> Controller Processed = 40  Output = 0
20 ns -> Output Signal = 0
20 ns -> Sensor Value = 60
25 ns -> Output Signal = 0
30 ns -> Sensor Value = 80
30 ns -> Output Signal = 0
35 ns -> Output Signal = 0
40 ns -> Controller Processed = 80  Output = 0
40 ns -> Output Signal = 0
40 ns -> Sensor Value = 100
45 ns -> Output Signal = 1
50 ns -> Sensor Value = 20
50 ns -> Output Signal = 1
55 ns -> Output Signal = 1
60 ns -> Controller Processed = 20  Output = 1
60 ns -> Output Signal = 1
60 ns -> Sensor Value = 40
65 ns -> Output Signal = 0
70 ns -> Sensor Value = 60
70 ns -> Output Signal = 0
75 ns -> Output Signal = 0
80 ns -> Controller Processed = 60  Output = 0
```

```
80 ns -> Sensor Value = 80
85 ns -> Output Signal = 1
90 ns -> Sensor Value = 100
90 ns -> Output Signal = 1
95 ns -> Output Signal = 1
===== Simulation Finished =====
```

Conclusion:

The SystemC model successfully simulates an embedded controller. Timing analysis helps evaluate response delay and verifies whether the controller satisfies real-time requirements.

Question 15: Implement a SystemC model for a multi-threaded embedded system and evaluate synchronization techniques.

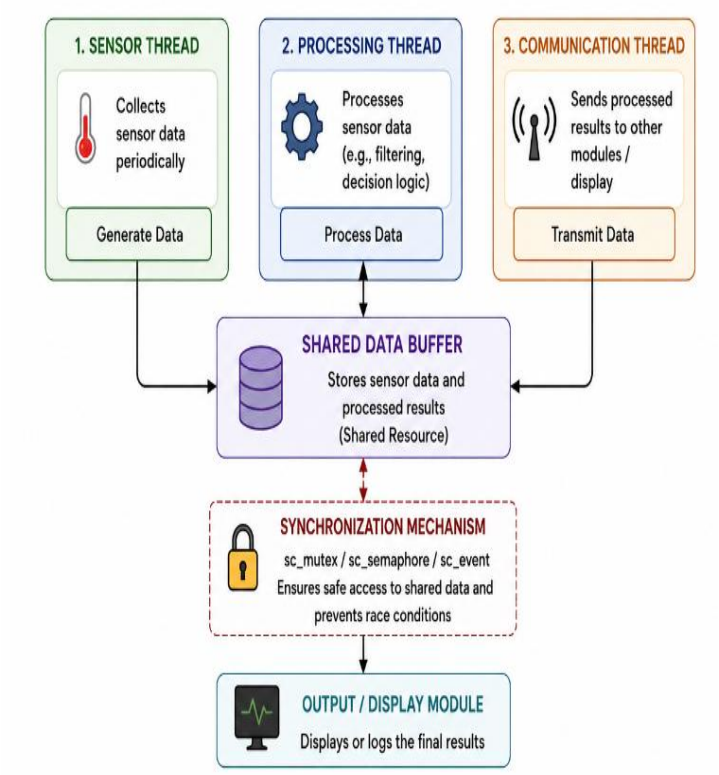
Problem Understanding:

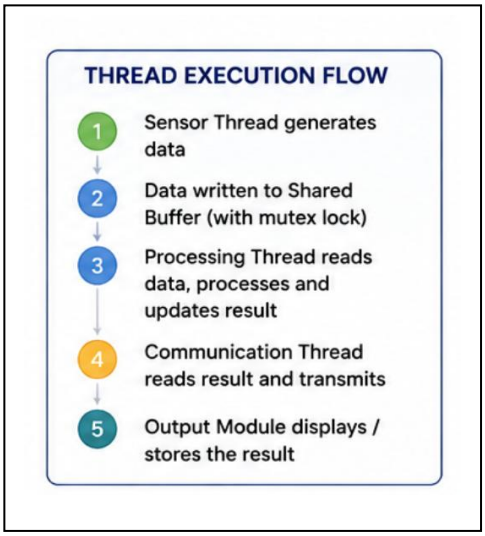
The objective is to design a **multi-threaded embedded system** where multiple tasks execute simultaneously and communicate safely.

Example:

- Sensor thread collects data.
- Processing thread analyzes data.
- Communication thread sends results.

System Architecture:





Technique	Purpose
Mutex	Prevents simultaneous access to shared resources
Semaphore	Controls resource access
Event	Synchronizes thread execution

SystemC program:

testbench.cpp

```
#include <systemc.h>
SC_MODULE(EmbeddedSystem)
{
    sc_mutex mutex;
    int shared_data;
    void sensor_thread()
    {
        int value = 10;
        while (true)
        {
            mutex.lock();

            shared_data = value;

            cout << sc_time_stamp()
                 << " -> Sensor Thread : Generated = "
                 << shared_data << endl;
            mutex.unlock();
            value += 10;
            if (value > 100)
                value = 10;
            wait(10, SC_NS);
        }
    }
    void processing_thread()
    {
        while (true)
        {
            wait(15, SC_NS);

            mutex.lock();

            int result = shared_data * 2;

            cout << sc_time_stamp()
                 << " -> Processing Thread : Result = "
                 << result << endl;

            mutex.unlock();
        }
    }
    void communication_thread()
    {
    {
        while (true)
        {
            wait(20, SC_NS);

            mutex.lock();

            cout << sc_time_stamp()
                 << " -> Communication Thread : Sending Data = "
                 << shared_data << endl;
            mutex.unlock();
        }
    }
    SC_CTOR(EmbeddedSystem)
    {
        shared_data = 0;

        SC_THREAD(sensor_thread);
        SC_THREAD(processing_thread);
        SC_THREAD(communication_thread);
    }
};
int sc_main(int argc, char* argv[])
{
    EmbeddedSystem system("EmbeddedSystem");

    cout << "===== Multi-Threaded Embedded System Simulation
=====" << endl;

    sc_start(100, SC_NS);

    cout << "===== Simulation Finished =====" << endl;

    return 0;
}
```

Output:

```
===== Multi-Threaded Embedded System Simulation =====
0 s -> Sensor Thread : Generated = 10
10 ns -> Sensor Thread : Generated = 20
15 ns -> Processing Thread : Result = 40
20 ns -> Sensor Thread : Generated = 30
20 ns -> Communication Thread : Sending Data = 30
30 ns -> Processing Thread : Result = 60
30 ns -> Sensor Thread : Generated = 40
40 ns -> Communication Thread : Sending Data = 40
40 ns -> Sensor Thread : Generated = 50
45 ns -> Processing Thread : Result = 100
50 ns -> Sensor Thread : Generated = 60
60 ns -> Processing Thread : Result = 120
60 ns -> Sensor Thread : Generated = 70
60 ns -> Communication Thread : Sending Data = 70
70 ns -> Sensor Thread : Generated = 80
75 ns -> Processing Thread : Result = 160
80 ns -> Sensor Thread : Generated = 90
80 ns -> Communication Thread : Sending Data = 90
90 ns -> Processing Thread : Result = 180
90 ns -> Sensor Thread : Generated = 100
===== Simulation Finished =====
```

Synchronization Evaluation

Without Synchronization:

- Race condition occurs.
- Data corruption may happen.

With Mutex:

- Only one thread accesses shared data at a time.
- Ensures data consistency.

Performance Analysis:

Parameter	Result
Thread Execution	Parallel
Data Safety	Improved
Response Time	Reduced

Conclusion:

The multi-threaded SystemC model demonstrates parallel task execution and synchronization using mutex techniques. Proper synchronization improves reliability and prevents race conditions in embedded systems.